

Cyber security attack recognition on cloud computing networks based on graph convolutional neural network and graphsage models

Fargana Abdullayeva^{*}, Suleyman Suleymanzade

Institute of Information Technology, B. Vahabzade Street, 9A, AZ1141, Baku, Azerbaijan

ARTICLE INFO

Keywords:

Cyber security
Graph neural networks
Ddos, Network attack detection
Graphsage
Cloud computing

ABSTRACT

In this paper, the modeling of the network attacks of cloud computing through Graph Neural Networks is considered. Based on structural features and relationships between neighboring nodes and the edges of the cloud ecosystem a cyberattack detection method is proposed. A simulation dataset is created on the CSE-CIC-IDS2018 dataset to train and test the proposed graph neural network based models. In a comparative analysis of the suggested method with the existing one superior results are obtained from the model constructed on the GraphSAGE algorithm. Thus in the recognition of dataset samples, the model obtained a value of 0.97739 according to the accuracy metric. The values obtained by the algorithm on precision, recall, and F1-score metrics were also higher compared to the Graph Convolutional Neural Network model.

1. Introduction

Recently, the number of cyberattacks has increased significantly [1]. Their growth has harmed the economy of countries around the world. They create continuous data breaches that threaten the privacy of hundreds of millions of users. Detecting these cyberattacks is considered an important issue.

Numerous approaches have been developed to detect cyberattacks. These approaches try to detect malicious traffic while keeping the genuine (benign) traffic as is. Here, the detection of malicious traffic is brought into the incoming traffic classification issues. Examples of these methods include autoencoders that use reconstruction errors as the deviation value [2], Generative Adversarial Networks [3], and methods based on LSTM models.

Detection of network cyberattacks is typically performed on flow network data. Most detection systems try to simplify the issue by looking at each flow separately. These methods do not consider the presence of multiflow attacks (such as DDoS). This assumption is not justified in most application areas. The detection system must be able to process multiple flows simultaneously. To meet this requirement, the network traffic flow must be modeled as a graph. Here, the flow comes from communication nodes such as IP address, port number, protocol, and the column fields of data contain network traffic flows such as the number of packets transmitted, number of bytes transmitted, duration of the flow, etc. This flow data can be represented in graph format. In this case, network flow nodes (IP address) are represented as graph nodes, and network traffic flows are represented as graph edges. Both topological information (features of nodes) and information reflected in edge features are important for network traffic classification and attack detection.

^{*} Corresponding author.

E-mail address: a_farqana@mail.ru (F. Abdullayeva).

Traditional neural network models cannot handle graph-structured data. Recently, GNN models have been developed and designed to enable neural networks to work with graph-structured data. Graph Convolutional NN, Gated Graph Sequence NN, Graph Attention Networks, and Message Passing NN are available graph neural network models.

In the existing methods, all samples of the database are involved in the process of training the neural network. This leads to memory error and requires a large computational cost (time consumption). In addition, traditional graph methods developed for attack detection are based on the graph structure of connections and use a graph measure called centrality. To construct this structure and to calculate the centrality measure, the data about the node must be available to the algorithm at the same time. In most cases, it is impossible to obtain detailed information about each traffic packet received by each node in the network.

In this paper, a cloud network cyberattack detection method based on the Graph Neural Network approach is proposed. Here, in the training iteration, it is assumed to use node representations in one package. This made it possible to sufficiently reduce the number of selected nodes and calculation costs, increase the speed of the algorithm, and increase the cyberattack detection accuracy.

The main scientific contributions of the paper are as follows:

- For the detection of cyberattacks on cloud networks, a method using topological nodes and their features is proposed;
- To evaluate the effectiveness of the approach, the proposed GNN and GraphSAGE based models are applied to the CSE-CIC-IDS2018 dataset.

The paper consists of the following sections.

Section 2 introduces the overview of related works. Section 3 describes the Graph Neural Network concept. Section 4 presents the graph modeling process of cloud network flows. Section 5 represents the suggested approach architecture. Section 6 provides experiments for the evaluation of the efficiency of proposed method. Section 7 presents the conclusion.

2. Related works

Numerous studies have been conducted in the direction of neural networks, deep learning models, and their hybridization and optimization to perform accurate detection of cyberattacks on cloud network environments. Graph-based approaches to identify malicious nodes and threads have also been proposed.

[4] proposed a model called E-GraphSAGE based on GNN for network intrusion detection in IoT environment. Here, the flow of information related to intrusions is described in graph format. The proposed method allows to obtain edge features and topological information for detecting network intrusions in the IoT environment. E-GraphSAGE performs graph edge classification to detect malicious network flows. [5] suggested a graph embedding method for network flow intrusion detection. The network flows were initially transformed into a first-order and second-order graph by the authors. The IP address and port number of a single host are used by the first-order graph to learn the latent features from that host's perspective. The second-order graph makes use of source IP addresses, source ports, destination IP addresses, and destination ports in an effort to learn the latent properties from a global standpoint. A Random Forest classifier is then trained to recognize network attacks using the extracted graph embeddings and the raw characteristics. [6] suggested a method for detecting P2P botnet nodes based on graph convolutional neural networks (GCNs). There first created botnet connections that were combined together to produce botnet traffic, then to classify the botnet nodes the GCN is used. The method simply takes into account the topological data of the network connectivity graph for P2P botnet node categorization. No flow or node attributes are included in the resulting graph. This method restricts the ability to detect botnet attacks. Additionally, it does not fully utilize the network flow characteristics or the information offered in network flow data.

For the detection of online spammers [7] proposed Deep Graph neural network-based Spammer detection (DeG-Spam) model. Predicting cyber threats is critical in preventing malicious attacks and preventing security risks. [8] proposed Dynamic Heterogeneous Graph training model for cyber security threat prediction. Here, cyber threat prediction is studied on Common Vulnerabilities and Exposures program's details (CVE) and ExploitDB open databases. Here, first of all, the interactive relationships between various objects that create a cyber threat are characterized with the help of a heterogeneous graph. Cyber threat prediction is then modeled as a dynamic link prediction problem. An embedding method was proposed to study the dynamic change of the graph.

Existing works proposed for detecting DDoS attacks do not exploit the interactions between packets and flows within the traffic. This information is considered important information that significantly increases the effectiveness of detection. [9] proposed a GNN-based approach for DDoS attack detection. The so-called GraphDDoS approach uses end-point traffic graphs. Here, traffic is converted into end-point traffic graphs. This graph contains information about packet interactions (single flow structure) and flow interactions (multiple flow structure). Then, the transformed end-point traffic graphs are passed to the GNN classifier and DDoS attack patterns are learned. The ability of the GraphDDoS model to learn graph-structured attack templates has been evaluated on various databases.

Traditional graph methods developed for detecting DDoS attacks are based on the graph structure of connections and use a graph measure called centrality. To construct this structure and to calculate the centrality measure, the algorithm must have all the data about the node available at the same time. In most cases, it is not possible to obtain detailed information about each traffic packet received by each node in the network. For this reason [10] believes that detection of DDoS attacks should be based on easily accessible information. For example, traffic volume. Here, the author proposed an approach based on the Spatio-Temporal Graph Convolutional Networks model for detecting DDoS attacks in a network environment.

[11] proposed a hierarchical approach based on the attention-based GNN model for detecting APT attacks. APT behaviors can be detected at both the system and network layers. The proposed approach is composed of three components. In the first block of the

Table 1
Comparative analysis of the related studies.

Reference	Type	Aim	Method	Features	Dataset
[4]	Edge classification	IoT malicious network flows detection	Proposes E-GraphSAGE model based on GNN	44 network flow features	ToN-IoT and BoT-IoT datasets
[12]	Node classification	Attack detection	Proposes a novel Graph Neural Network model	packet size and the inter-arrival times	CIC-IDS2017 dataset
[9]	Node classification	Endpoint traffic Graph classification	Proposes GraphDDoS model based on GNN	packet relationships flow relationships	DDoS attacks dataset
[11]	Node classification	System-level and network-level APT attack detection	Attention-based GNN modeline əsaslanan iyerarxik yanaşma təklif edilib.	System audit logs from different operating systems Network events from various protocols	Stream-Spot and LANL datasets
Proposed method	Node classification	Cybersecurity attack detection in cloud computing network	Cybersecurity attack detection system based on GCN and GraphSAGE model	80 features of cloud network flow	CSE-CIC-IDS2018 dataset

model, a graph is constructed, in the second block, the nodes of the graph are transformed into a small-sized vector (latent vector), and in the third block anomalies are detected. The proposed method was tested on Stream-Spot and LANL databases for detection of APT attacks at both system and network layers.

The GraphSage algorithm, a popular GNN model, performs node classification using node attributes. In this model, classification of edges using edge features is not envisaged. To overcome this problem, [12] proposed the E-GraphSAGE algorithm. In this algorithm, information related to edges (flow) was used in the process of describing the graph. The use of this information allows the transformation of edges into latent vectors and the classification of edges. Therefore, network flows are classified into benign and attack classes.

The paper in [13] proposes its minbatch based GNN version as ours to obtain a training procedure that is more able to apply in the complex Industrial Internet environment. Nevertheless, our GraphSAGE represents a more valid algorithm compared with this one, as this derived version produces sometimes inferior results. This approach incorporates a pre-sampling step prior to model training, leading to the creation of smaller graphs and improved scalability. However, our approach does not consider the issue of dividing a large graph into small parts.

Comparative analysis of the related studies are described in Table 1.

Fuzzy modeling and control-based approaches are also used to increase the resilience of cloud networks exposed to attacks. Using sophisticated fuzzy modeling and control techniques can improve communication systems' resilience to deception attacks. Essentially, communication systems can be made more resilient and adaptable to deception attacks by utilizing sophisticated fuzzy modeling and control techniques, which will ultimately increase overall performance and security. The following are mitigation strategies for deception attacks [14,15]:

Anomaly detection. The use of fuzzy logic can help in the development of systems that are capable of detection suspicious behaviors or patterns in communication networks that could point to the existence of deception attacks. Such attacks can be quickly identified so that suitable countermeasures can be put in place to lessen their impact.

Adaptive filtering. Adaptive filtering systems that dynamically respond to system changes brought about by deception attacks can be designed using fuzzy logic. By reducing or eliminating the consequences of misleading signals, these filters can enhance system performance as a whole.

Behavior modeling. The communication system's typical behavior as well as potential attackers' actions can be modeled using fuzzy logic. Effective detection and response to deception attacks can be achieved by applying fuzzy inference algorithms to analyze deviations from the predicted behavior.

Applications of game theory. The interactions between attackers and defenders in communication networks can be modeled by combining game theory and fuzzy logic. Strategies to effectively repel deception attacks while reducing the impact on legitimate communication can be developed by utilizing fuzzy game theory.

Service allocation. In communication systems, fuzzy control approaches can maximize service allocation, guaranteeing that services are distributed effectively even when deception attacks are present. This can entail adjusting bandwidth, power, or other resources spontaneously to minimize the effects of attacks and preserve system functionality.

Feedback management. Fuzzy control systems can be employed to create feedback mechanisms that constantly assess the communication system's effectiveness and make real-time adjustments to the system's parameters to mitigate the impact of deception attacks.

Integration with machine learning. To enhance machine learning methods for intrusion detection and categorization, fuzzy modeling can be used. Communication systems can become more resilient over time by learning to adjust to novel attack patterns by fusing fuzzy logic with machine learning techniques.

3. Graph neural network

A graph is an edge-and vertex-based non-linear data structure. In a network, the edges are the lines or arcs that join any two nodes, while the vertices are sometimes referred to as nodes. A matrix of adjacency is employed to depict the graph. A graph can be

represented as a matrix of boolean (0 s and 1 s) using an adjacency matrix. Without numerical data, a machine learning model is unable to learn from observations. The input observations for this model must first be converted into a numerical representation that can be learned. This numerical format is called a vector. A vector is a list of numbers.

Assume that the graph $G = (V, E)$ is given. V - is a set of nodes, E - is a set of edges. $A \in R^{N \times N}$ adjacency matrix, N - total number of nodes, $X \in R^{N \times C}$ node attribute (feature matrix), C - is the number of features for each node. The goal of the graph neural network is to learn the effective representations of the node ($H \in R^{N \times F}$, F - is the size of the representations of the node) by combining the structural information of the graph and the attributes of the node. Graph representation learning, also known as Graph embedding, is an approach that is capable of transforming graph nodes into vectors with a lower dimension (latent vector) while maximally preserving structural and semantic information. These representations are then used in the node classification process. The goal of GNN is to iteratively update the representations of a node by combining the representations of a node's neighbors and their own representations.

GNN perform two important functions [16]:

1. Aggregate. It aggregates the information of each node's neighbors.
2. Combine. Updates node representations by gathering aggregated information from neighbors.

A Graph Neural Network is mathematically modeled in the general case as follows.

Assume that the initial representation of a node is $H^0 = X$. K - is the total number of layers of GNN. The Aggregation and update functions for $k = 1, 2, \dots, K$ are expressed as follows.

$$a_v^k = \text{AGGREGATE}^k \{ H_u^{k-1} : u \in N(v) \} \quad (1)$$

$$H_v^k = \text{COMBINE}^k \{ H_v^{k-1}, a_v^k \} \quad (2)$$

where $N(v)$ is the set of neighbors of the v -th node. The final representations are reflected in the H^k node representations found in the last layer.

Node representations can be utilized for node classification after they are acquired. The v -th node's label, represented by the $\hat{y}_v$ in the node classification task, can be found by applying the Softmax function:

$$\hat{y}_v = \text{Softmax}(WH_v^T)$$

where $W \in R^{|Y| \times F}$ is the output space $|Y|$'s number of labels.

The whole model is trained with the requirement of minimizing the subsequent loss function in the case of labeled nodes.

$$O = \frac{1}{n} \sum_{i=1}^{n_l} \text{loss}(\hat{y}_i, y_i) \quad (3)$$

where y_i is the label of the i -th node, n_l is the number of labeled nodes, $\text{loss}(\cdot, \cdot)$ is a loss function (for example, the crossentropy loss function). The neural network in general is optimized by minimizing the objective function O .

The general working principle of GNN was described above. The most common types of GNN are the Graph Convolutional Network and GraphSAGE.

Softmax is an activation function that scales numbers (logits) into probabilities. The output of a Softmax is a vector (for example, y) with probabilities of each possible outcome. The probabilities in vector y sums to one for all possible outcomes or classes.

Softmax mathematically is defined as:

$$S(y)_i = \frac{\exp(y_i)}{\sum_{j=1}^n \exp(y_j)}$$

where, y is an input vector to a softmax function, S . It consist of n elements for n classes. y_i the i -th element of the input vector. $\exp(y_i)$ exponential function applied on y_i . $\sum_{j=1}^n \exp(y_j)$ - normalization term. It ensures that the values of output vector $S(y)_i$ sums to 1 for i -th class and each of them is in the range 0 and 1 which makes up a valid probability distribution. n - number of classes.

The cross-entropy loss function measures the difference between the true probability distribution and the predicted probability distribution. Given that the predicted distribution differs greatly from the true distribution in this instance, the cross-entropy loss would be larger. The cross-entropy loss function has the following mathematical expression:

$$L(\hat{y}, y) = - \sum_k^K y_k \log(\hat{y}_k)$$

where, $\hat{y}$ is the predicted probability distribution, y is the true probability distribution, and the summation is taken over all classes. y_k is 0 or 1.

Table 2
Network flows in cloud computing.

Flow	Source IP	Destination IP	Feature 1	...	Feature d
f(1)	172.143.1.1	172.188.1.1			
f(2)	172.168.3.4	172.188.1.2			
f(3)	172.168.3.4	172.188.1.1			

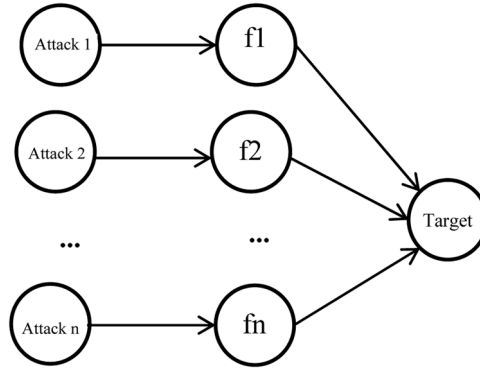


Fig. 1. DDoS attack graph representation.

3.1. Graph convolutional networks

In Graph Convolutional Networks, the node representations are updated at each layer based on the following propagation rule:

$$H^{k+1} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^k W^k\right) \quad (4)$$

where $\tilde{A} = A + I$ is the adjacency matrix of the given undirected graph G , I is the unity matrix. $\tilde{D}$ is the diagonal matrix and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. $\sigma(\cdot)$ is the activation function, which can be ReLu or Tanh. $W^k \in R^{F^k \times F^{k+1}}$ is a linear transformation matrix, trained in the optimization phase. Here, F^k denotes the size of F^k node representations in k -th and $(k+1)$ -th layers, respectively.

3.2. GraphSAGE neural network

GraphSAGE is an extension of the graph convolutional network.

Assume that a graph $G(V, E)$ is given. $N(v)$ is a neighbor of node v , while h is the representation of the node, the node's mapping is calculated by the following equation:

$$h_{N(v)}^{(l+1)} = \text{AGGREGATE}_l(\{h_u^{(l)}, \forall u \in N(v)\}) \quad (5)$$

While average aggregator is used in Graph Neural Network, GraphSAGE algorithm uses LSTM aggregator and Pooling aggregator to aggregate information from neighbors. Another difference of the algorithm with GNN is that it uses a concatenation function instead of a summation function to combine the information of the target node and its neighbors. This operation is expressed as follows:

$$h_v^{(l+1)} = \sigma\left(W^{(l+1)} \cdot \text{CONCAT}\left(h_v^{(l)}, h_{N(v)}^{(l+1)}\right)\right) \quad (6)$$

where $W^{(l+1)}$ is the weight matrix, σ is the activation function.

A mini-batch training strategy is provided in the GraphSage algorithm to enable GNN to work on large-scale graphs. This strategy allows to reduce the computational costs in the training phase. In each training iteration, only the nodes used to compute the representations in the packets are considered. This operation significantly reduces the number of selected nodes. In order to increase the training efficiency, the strategy of selecting the specified number of neighbors was also applied in the GraphSAGE algorithm. GraphSAGE is an algorithm that can perform inductive representation learning on large graphs. In order to speed up the training process in this algorithm, functions of generalized aggregator, mini-batch training, selection of specified number of neighbors are provided.

4. Graph modeling of cloud network flows (host connectivity graph)

Assume a flow $f \in F$ consisting of S source host and D - destination host is given (Table 2).

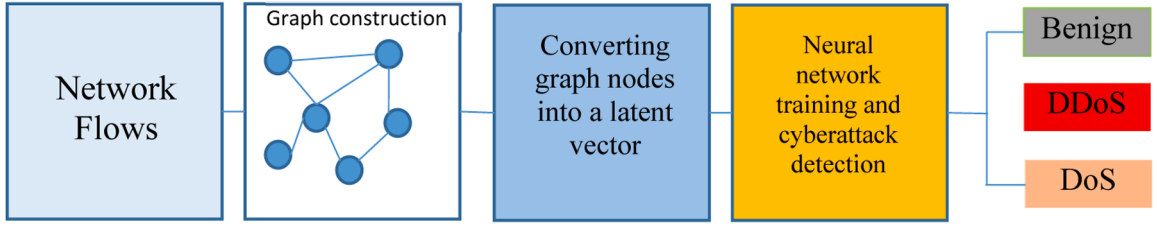


Fig. 2. System for detecting cyber security attacks using GNN models.

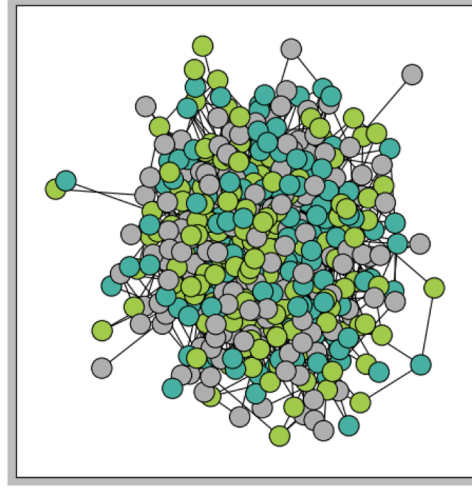


Fig. 3. Graph topology of CSE-CIC-IDS2018 dataset.

In this case, the host may be the one transmitting (Source IP) or receiving (Destination IP) traffic, and the graph's nodes correspond to each flow.

Every attack has a different pattern of flow. They are easily distinguishable from one another thanks to these flow patterns. A DDoS attack's topology is represented as a graph with numerous connections from various hosts to the same target since DDoS attacks are distributed. Fig. 1 depicts a graphical representation of a multiflow DDoS attack, where the attackers are denoted by Attack_n , target nodes are denoted by Target, the individual flows that were employed to carry out the attack denoted by f_n .

5. Suggested methodology

A suggested methodology based on a GNN model for identifying cyber attacks in a cloud computing network environment is illustrated in Fig. 2

Here, information is exchanged between nodes to train the neural network model. To make the last prediction, flows are converted into latent vectors (embedding). The information flow of the proposed model is represented as follows:

- Step 1.** Transfer of sequence of packets which is represented as network flow from a source computer to a destination is performed.
- Step 2.** By capturing the similarities among data points, which is characterized by an adjacency matrix and an edge attribute matrix a graph structure construction is provided.
- Step 3.** Converting graph nodes into a latent vector is conducted. Transformation nodes, edges, and their characteristics into vector space with a reduced dimension while retaining the graph structure.
- Step 4.** Neural network training and cyberattack detection. Model leverages Deep Neural Network classifier to detect cyberattacks from the new representations. GNN is trained with network traffic using latent features, which includes Benign traffic, DDoS and DoS attacks.

6. Experiments

The data gathered from the network is supplied into the input of a graph neural network. The rows of the dataset (IP addresses) constitute the graph's nodes, and the information exchanged between these nodes is represented by the edges. The features of the dataset that are assigned to the nodes in the graph topology determine how the nodes interact with one another. Dataset columns

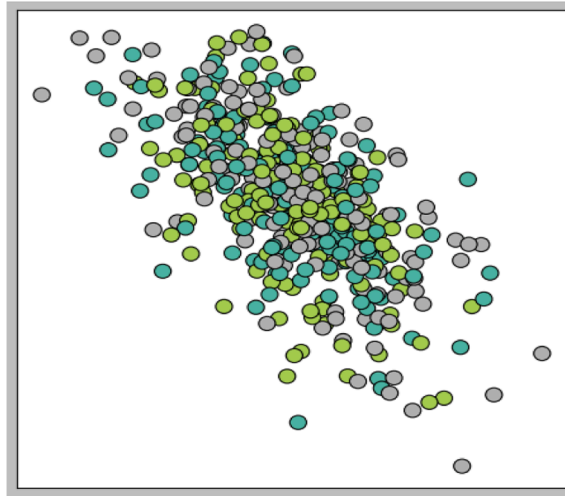


Fig. 4. GCN embedding visualization.

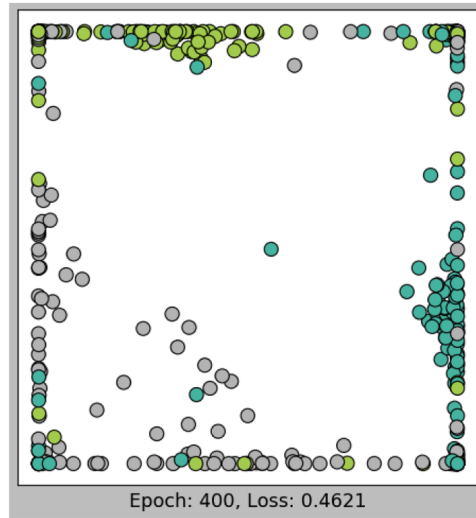


Fig. 5. GCN embedding visualization in 400th iteration.

consist of multiple fields. These fields include protocols, IP addresses, their associated ports, the total number of bytes and packets transferred, duration, the start time of each flow, etc. Source nodes of constructed graph is the network layer source IP address, edge information is the remaining traffic information.

Each node in the graph is given a feature from the dataset, establishing a relationship between them. The suggested method's efficacy was assessed on the simulation dataset generated on the CSE-CIC-IDS2018 dataset [17]. Here, the graph contains three types of nodes: Benign, DoS, and DDoS. The features assigned to the nodes allow to classify of the graph nodes into three classes.

The number of nodes participating in the network is 498, the number of edges is 2952, the number of features is 80, and the number of classes is 3. 80 percent of the data was used for training and 20 percent for testing. Thus, out of 498 samples of the database, 398 samples were used for training and 100 samples were used for testing. The issue of detection of DoS and DDoS attacks was considered in this work. The dataset's source and destination data are used to construct multiple connections between nodes. The features of the nodes of the presented graph are described in the form of a feature vector. The number of packets transmitted from one node (edge) to another in a certain time period, the number of received packets, the number of bytes transmitted from one node (edge) to another in a certain time period, the number of bytes received by the node, the duration of the connection between two nodes when exchanging data, etc. form the features of the vector.

Fig. 3 represents the graph topology of the CSE-CIC-IDS2018 dataset.

Embedding visualization of the GCN in 1st iteration is represented in Fig. 4.

In the experimental study of the Graph Convolutional Network model, the loss function is taken CrossEntropy, the optimization

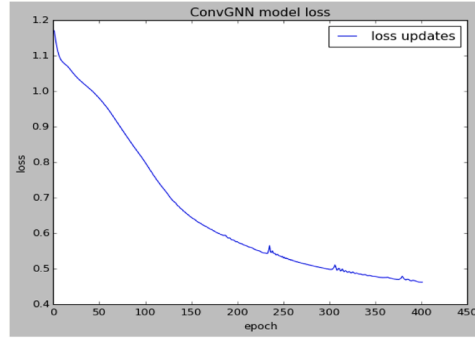


Fig. 6. ConvGNN loss visualization.



Fig. 7. GraphSAGE embedding visualization in 400th iteration.

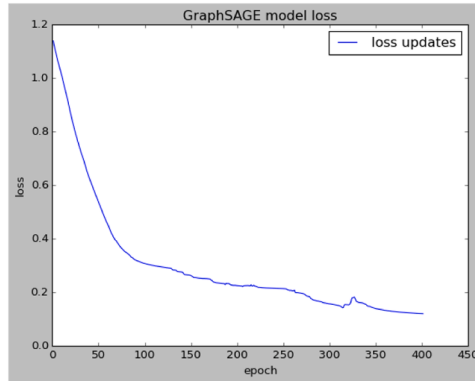


Fig. 8. GraphSAGE loss visualization.

function is Adam, and the number of iterations is 400. The GCN embedding visualization in the 400th iteration is represented in Fig. 5

The GCN model was successfully trained on the dataset, with a loss of 0.4621. This scene is depicted in Fig. 6.

In the training of the GraphSAGE algorithm, the optimization function is taken Adam, the loss function CrossEntropy, and the learning rate 0.01. GraphSAGE embedding visualization in 400th iteration is depicted in Fig. 7.

The GraphSAGE model was successfully trained on the dataset, with a loss of 0.1194. This scene is depicted in Fig. 8.

The efficacy of the suggested approach was assessed using the F1-score, Loss functions, Accuracy, Precision, and Recall metrics. Table 3 presents the evaluation results.

As can be seen from Table 3, the GraphSAGE model has reduced loss in dataset sample recognition when compared to the Graph Convolutional Neural Network model. Thus, the loss value of the GCN model was 0.4621, and the loss value of the GraphSAGE model was 0.1194. Further evaluation measures also showed good performance of the GraphSAGE algorithm. This algorithm obtained a value

Table 3

Assessment of the efficacy of techniques for detecting cyberattacks.

Techniques	Class	Accuracy	Loss	Precision	F1-score	Recall
Graph Convolutional Neural Network	Benign	0.80151	0.4621	0.96	0.83	0.74
	DoS			0.79	0.78	0.76
	DDoS			0.65	0.78	0.99
GraphSAGE	Benign	0.97739	0.1194	0.99	0.98	0.97
	DoS			0.95	0.97	0.98
	DDoS			0.99	0.98	0.98

of 0.97739 according to the accuracy metric. The values obtained by the algorithm on precision, recall, and F1-score metrics were higher compared to the GCN model.

7. Conclusion

The reason to use graph neural network approach to detect DoS attacks in this paper is that, in the case of detection of cyber attacks in the network infrastructure of cloud systems, the detection of network cyber attacks is usually performed on streaming network data. Most detection systems try to simplify the issue by looking at each stream separately. These methods do not consider the presence of multi-stream attacks (e.g. DDoS) and this assumption is not justified in most application areas. The detection system must be able to process multiple streams simultaneously. To meet this requirement, the network traffic flow must be modeled as a graph. Here, the flow comes from communication nodes such as IP address, port number, protocol, and the column fields of data are number of packets transmitted, number of bytes transmitted, duration of the flow, etc. such as network traffic flows. This flow data can be represented in graph format. In this case, network flow nodes (IP address) are described as graph nodes, and network traffic flows are described as graph edges. Topological information, including information reflected in language features, is important for network traffic classification and attack detection.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgment

This work was supported by the Science Foundation of the State Oil Company of Azerbaijan Republic (Contact No. 3 LR-AMEA).

References

- [1] Jason F. Cyber Security Statistics The Ultimate List Of Stats, Data & Trends. <https://purplesec.us/resources/cyber-security-statistics/>; 2021.
- [2] Abdullayeva FJ. Cloud computing virtual machine workload prediction method based on variational autoencoder. *Int J Syst Software Secur Protection (IJSSSP)* 2021;12(2):33–45.
- [3] Li D, Chen D, Jin B, Shi L, Goh J, Ng SK. MAD-GAN: multivariate anomaly detection for time series data with generative adversarial networks. In: *Proc. of the International Conference on Artificial Neural Networks*. Springer; 2019. p. 703–16.
- [4] Lo WW, Layeghy S, Sarhan M, Gallagher M, Portmann M. E-GraphSAGE: a Graph Neural Network based Intrusion Detection System for IoT. In: *Proc. of the IEEE/IFIP Network Operations and Management Symposium*; 2022. p. 1–9.
- [5] Xiao Q, Liu J, Wang Q, Jiang Z, Wang X, Yao Y. Towards network anomaly detection using graph embedding. In: *Proc. of the International Conference on Computational Science*; 2020. p. 156–69.
- [6] Zhou J., Xu Z., Rush A., Yu M., Automating Botnet Detection with Graph Neural Networks, 2020, pp. 1–8. arXiv:2003.06344v1 [cs.CR].
- [7] Guo Z, Tang L, Guo T, Yu K, Alazab M, Shalaginov A. Deep Graph neural network-based spammer detection under the perspective of heterogeneous cyberspace. *Future Generation Computer Systems* 2021;117:205–18.
- [8] Zhao J, Shao M, Wang H, Yu X, Li B, Liu X. Cyber threat prediction using dynamic heterogeneous graph learning. *Knowl Based Syst* 2022;240:1–14.
- [9] Li Y, Li R, Zhou Z, Guo J, Yang W, Du M, Liu Q. GraphDDoS: effective DDoS Attack Detection Using Graph Neural Networks. In: *Proc. of the IEEE 25th International Conference on Computer Supported Cooperative Work in Design*; 2022. p. 1275–80. 04-06 May.
- [10] Xie Q, Huang Z, Guo J, Qiu W. Spatio-Temporal Graph Convolutional Networks for DDoS Attack Detecting. *Machine Learning for Cyber Security*. In: *Proc. of the third International Conference on Machine Learning for Cyber Security*; 2020. p. 153–9. October 8–10.
- [11] Li Z, Cheng X, Sun L, Zhang J, Chen B. A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks. *Security and Communication Networks* 2021;2021:1–14. <https://doi.org/10.1155/2021/9961342>.
- [12] Pujol-Perich D., Su ' arez-Varela J., Cabellos-Aparicio A., Barlet-Ros P., Unveiling the potential of Graph Neural Networks for robust Intrusion Detection, 2021, arXiv:2107.14756v1 [cs.CR], pp. 1–7.
- [13] Lan J., Lu J.Z., Wan G.G., Wang Y.Y., Huang C.Y., Zhang S.B., Huang Y.Y., Ma J.N. E-minBatch GraphSAGE: an industrial internet attack detection model, *Security and communication networks*, Vol. 2022, pp. 1–12. <https://doi.org/10.1155/2022/5363764>.

- [14] Fatemeh A, William T, Emma F, Maria K. Detection and mitigation of deception attacks on cloud-based industrial control systems. In: Proc. of the 25th IEEE conference on innovation in clouds, internet and networks; 2022. p. 106–10.
- [15] Oluoha OU, Yange TS, Okereke GE, Bakpo FS. Cutting Edge Trends in Deception Based Intrusion Detection Systems—A Survey. *Journal of Information Security* 2021;12(4):250–69.
- [16] Wu L, Cui P, Pei J, Zhao L. Graph neural networks: foundations, frontiers, and applications. Springer Nature Singapore Pte Ltd.; 2022. p. 689.
- [17] CSE-CIC-IDS2018 on AWS dataset, Accessed on 25 september 2022. <https://www.unb.ca/cic/datasets/ids-2018.html>.